

Two Techniques:

The Potential Method

Divide and Conquer

Savage Chickens

by Doug Savage



www.savagechickens.com



"WE ALREADY HAVE QUITE A FEW PEOPLE WHO KNOW HOW TO DIVIDE. SO ESSENTIALLY, WE'RE NOW LOOKING FOR PEOPLE WHO KNOW HOW TO CONQUER."

Last time:

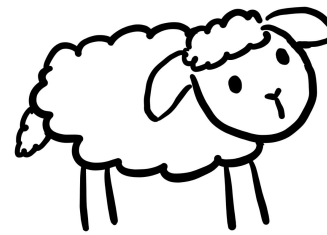
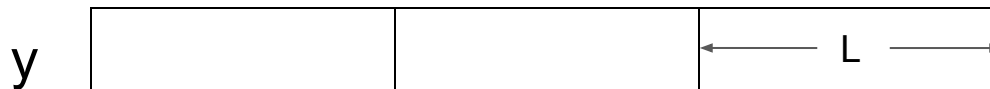
The Tiling problem (aka gcd)

Input: n-digit integers $x \geq y \geq 0$, but not both = 0.

recall: $x, y < 10^n$

Output: largest integer L that divides both x and y (aka greatest common divisor)

In other words: largest integer tile size that can exactly tile a path of length x and a path of length y



I want to tile two paths but only buy one size of tile

Euclid's idea!

$$\gcd(x, y) = \gcd(x-y, y)$$

$$= \gcd(x-2y, y)$$

$$= \gcd(x-3y, y)$$

$$= \gcd(x-ky, y) \text{ for any } k \text{ such that } x-ky \geq 0$$

$$= \gcd(x \bmod y, y)$$

Now we have a smaller instance of the same problem.

What should we do? How do we turn this **identity** into a **program** for computing gcd?

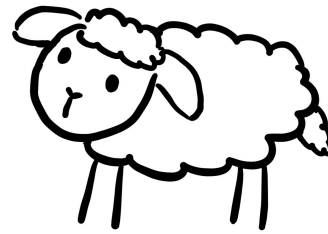
Euclid's Algorithm

Euclid(x,y): // for integers $x \geq y \geq 0$ (not both 0)

If $y = 0$: **return** ____ // base case

Else: **return** Euclid() // recursive case

This is called
pseudocode!
It's the language
we use to write
algorithms.



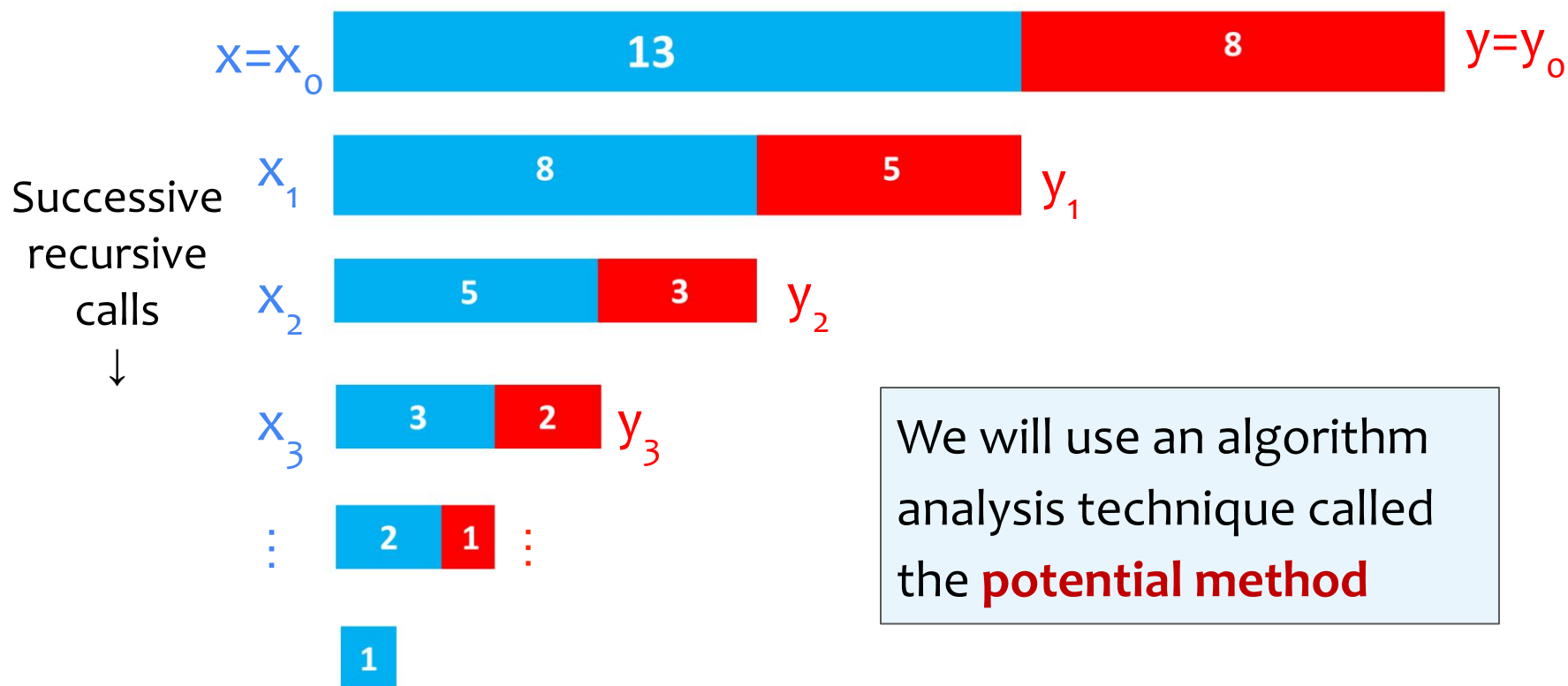
Analyzing Algorithms

In general, analyzing an algorithm requires two components:

- Proving correctness.
- Proving a running time bound.

Euclid's Algorithm: Running Time

We will show that, in each recursive call to Euclid, the x, y arguments are *collectively* decreasing *very quickly*



The Potential Method

Today we will analyze the running time of Euclid's algorithm using the **potential method**.

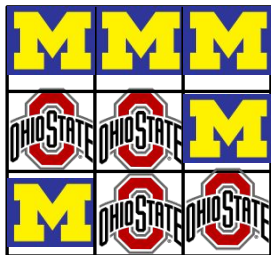
... But first, a toy example to illustrate this method

The potential method appeared in a movie!

<https://www.youtube.com/watch?v=oCoKdZXldyU>

A Flipping Game

- 3 x 3 board covered with two-sided chips:  / 
- Two players, **R (row)** and **C (column)**, alternately perform “flips”:
 - **R** flips every chip in some **row** with $\# \text{ Ohio State } > \# \text{ M }$
 - **C** flips every chip in some **column** with $\# \text{ Ohio State } > \# \text{ M }$
- If no flip is possible, then the game ends.
- **Question:** Must the game always end?



R flips
row 3



C flips
column 1



Let's formalize this reasoning into a general-purpose method

Intuitively, a **potential function argument** says:

If I start with not too much water in a leaky bucket, then water must stop leaking out.



Ingredients of the argument:

1. Define the “unit of time” e.g. one iteration of an algorithm
2. Define the **potential function S_i** $\leftarrow$ “amount of water in bucket” at time i
3. Provide an upper bound on S_0 .
4. Provide a lower bound on S_i (often 0).
5. Prove that the bucket “leaks quickly”. I.e. that S_i decreases by at least some fixed amount per unit time.
6. Use this to upper bound the total number of units of time.

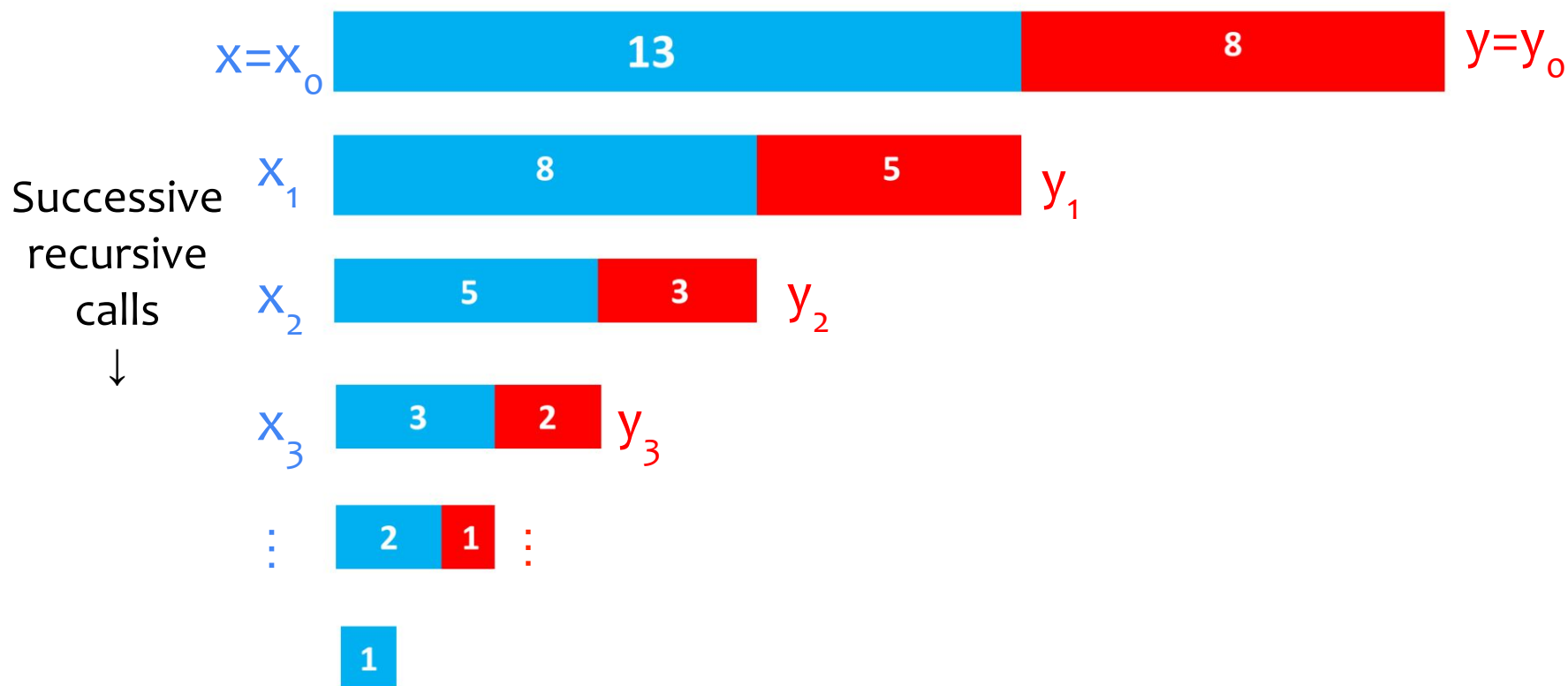
Analyzing the Flipping Game via a Potential Function

1. Unit of time = one player's turn.
2. Define the **potential function** $S_i = \#$  chips at turn i .
3. $S_0 \leq 9$.
4. $S_i \geq 0$ for all i .
5. At every turn the value of S_i decreases by at least **1**, i.e.: $S_{i+1} \leq S_i - 1$
6. This implies that the total number of turns is at most $S_0 \leq 9$.

Now let's apply the potential method to Euclid's Algorithm...

Euclid's algorithm: Running Time

We will show that, in each recursive call to Euclid, the x, y arguments are *collectively* decreasing *very quickly*



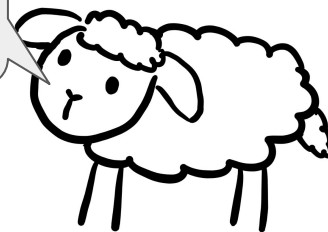
Analyzing Euclid's Algorithm via a Potential Function

1. Unit of time = one recursive call.
2. Define the **potential function** $S_i = y_i$.
3. $S_0 = y$.
4. $S_i \geq 0$ for all i .
5. At every recursive call the value of S_i decreases by at least **1**. Why?
6. Thus, the total number of calls to Euclid is at most $S_0 = y$.

But we already knew this! Recall that the brute-force algorithm from last lecture already achieved y iterations.

This is a ba-a-a-a-d
analysis of a good
algorithm

Conclusion: We need a function S_i that decreases by more.



Let's convince ourselves that potential functions $S_i = y_i$ and $S_i = x_i$ give $\exp(n)$ running time bounds
(for time unit = 1 recursive call)

$S_i = y_i$: What is an example of x, y values (≥ 10) such that S_i decreases by only 1, i.e. $y - y_1 = 1$?

$S_i = x_i$: What is an example of x, y values (≥ 10) such that S_i doesn't decrease at all, i.e. $x = x_1$?

Analyzing Euclid's Algorithm via a Potential Function

Finding the right potential function can be a fine art.

It turns out that even though neither $S_i = y_i$ nor $S_i = x_i$ work, $S_i = x_i + y_i$ does!

1. Unit of time = one recursive call.
2. Define the **potential function** $S_i = x_i + y_i$.
3. $S_0 = x + y$.
4. $S_i \geq 1$ for all i .
5. **Claim 1.** At every recursive call the value of S decreases by at least a **multiplicative factor**, specifically $S_{i+1} \leq (2/3) \cdot S_i$ for all i . (need to prove)
6. **Claim 2.** Claim 1 implies: total # recursive calls is $O(n)$. (need to prove)

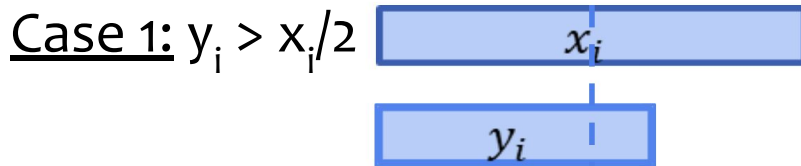
Consequence of Claim 2:

poly(n) by grade-school algorithm

Final running time = $O(n) \cdot (\text{time to compute } x \bmod y)$

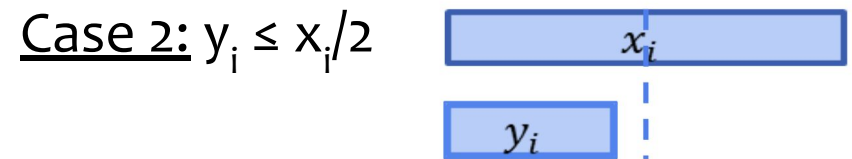
Analyzing Euclid's Algorithm via a Potential Function

Goal: For some $c < 1$, for all i we have $S_{i+1} \leq c \cdot S_i$.



$$\begin{aligned} S_{i+1} &= x_{i+1} + y_{i+1} \\ &= y_i + (x_i - y_i) \\ &= x_i \end{aligned}$$

$$\begin{aligned} S_i &= x_i + y_i \\ &> (3/2) \cdot x_i \end{aligned}$$



$$\begin{aligned} S_{i+1} &= x_{i+1} + y_{i+1} \\ &= y_i + x_i \bmod y_i \\ &< 2y_i \end{aligned}$$

$$\begin{aligned} S_i &= x_i + y_i \\ &\geq 2y_i + y_i \\ &= 3y_i \end{aligned}$$

In both cases: $S_{i+1} \leq (2/3) \cdot S_i$

Analyzing Euclid's Algorithm via a Potential Function

Claim 2: If $S_{i+1} \leq (2/3) \cdot S_i$ for all i , then total number k of recursive calls is $O(n)$.

Proof. Observe $S_k \geq 1$.

$$\text{So, } 1 \leq S_k \leq (2/3)^k \cdot (x+y)$$

$$\Leftrightarrow (3/2)^k \leq (x+y)$$

$$\begin{aligned} \Leftrightarrow k &\leq \log_{3/2}(x+y) \\ &< \log_{3/2}(2 \cdot 10^n) \\ &= O(n) \end{aligned}$$

$$\begin{aligned} S_0 &= x+y, \\ S_1 &\leq (2/3) \cdot (x+y), \\ S_2 &\leq (2/3)^2 \cdot (x+y), \\ &\dots \\ S_i &\leq (2/3)^i \cdot (x+y) \\ &\dots \\ S_k &\leq (2/3)^k \cdot (x+y) \end{aligned}$$

Recap: Analyzing Euclid's Algorithm via a Potential Function

Finding the right potential function can be a fine art.

It turns out that even though neither $S_i = y_i$ nor $S_i = x_i$ work, $S_i = x_i + y_i$ does!

1. Unit of time = one recursive call.
2. Define the **potential function** $S_i = x_i + y_i$.
3. $S_0 = x + y$.
4. $S_i \geq 0$ for all i .
5. **Claim 1.** At every recursive call the value of S decreases by at least a **multiplicative factor**, specifically $S_{i+1} \leq (2/3) \cdot S_i$ for all i . (proved!)
6. **Claim 2.** Claim 1 implies: total # recursive calls is $O(n)$. (proved!)

Consequence of Claim 2:

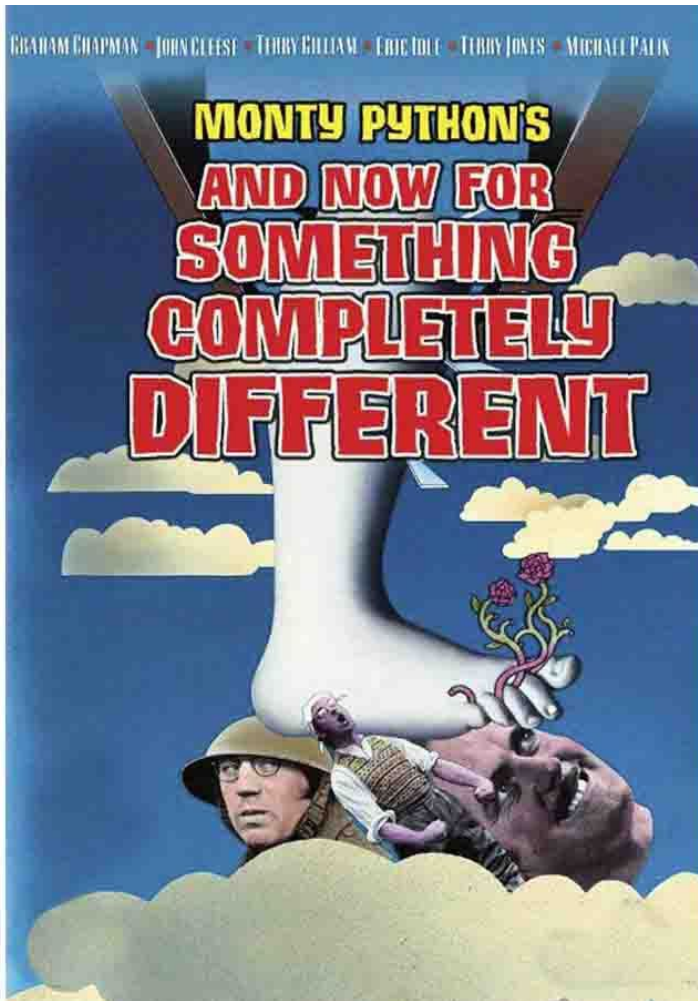
poly(n) by grade-school algorithm

Final running time = $O(n) \cdot (\text{time to compute } x \bmod y)$

When to use the potential method

Part of the challenge (and fun) of algorithm design + analysis is figuring out when to use which technique.

General intuition: The potential method could be useful when some quantity seems to be increasing or decreasing over the execution of the algorithm, getting you closer and closer to termination.



A Design
Technique:
Divide and
Conquer

Overview: Divide-and-Conquer Algorithms

Main Idea:

1. **Divide** the input into smaller sub-problems
2. Solve each sub-problem *recursively*
3. **Conquer**: Combine the sub-solutions to get a solution for the original input

Designing the Algorithm + Proving Correctness: an “art”

- Depends on problem structure, ad-hoc, creative

Running time Analysis: “mechanical”

- Express runtime using a recurrence
- Can often solve using the “Master Theorem”

Mergesort

Input:
Array of
numbers

1	2	3	4					n
6	0	4	6	...	...	...	...	3

Output:
Sorted

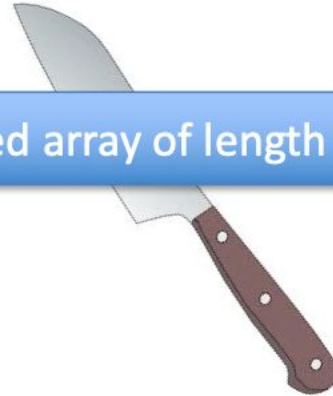
1	2	3	4					
0	3	4	6	6	...	...	...	...

Discovered by John von Neumann in 1945



Mergesort

Unsorted array of length n



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

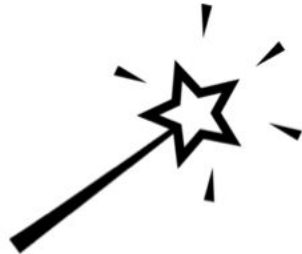
Unsorted array of length $n/2$

Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

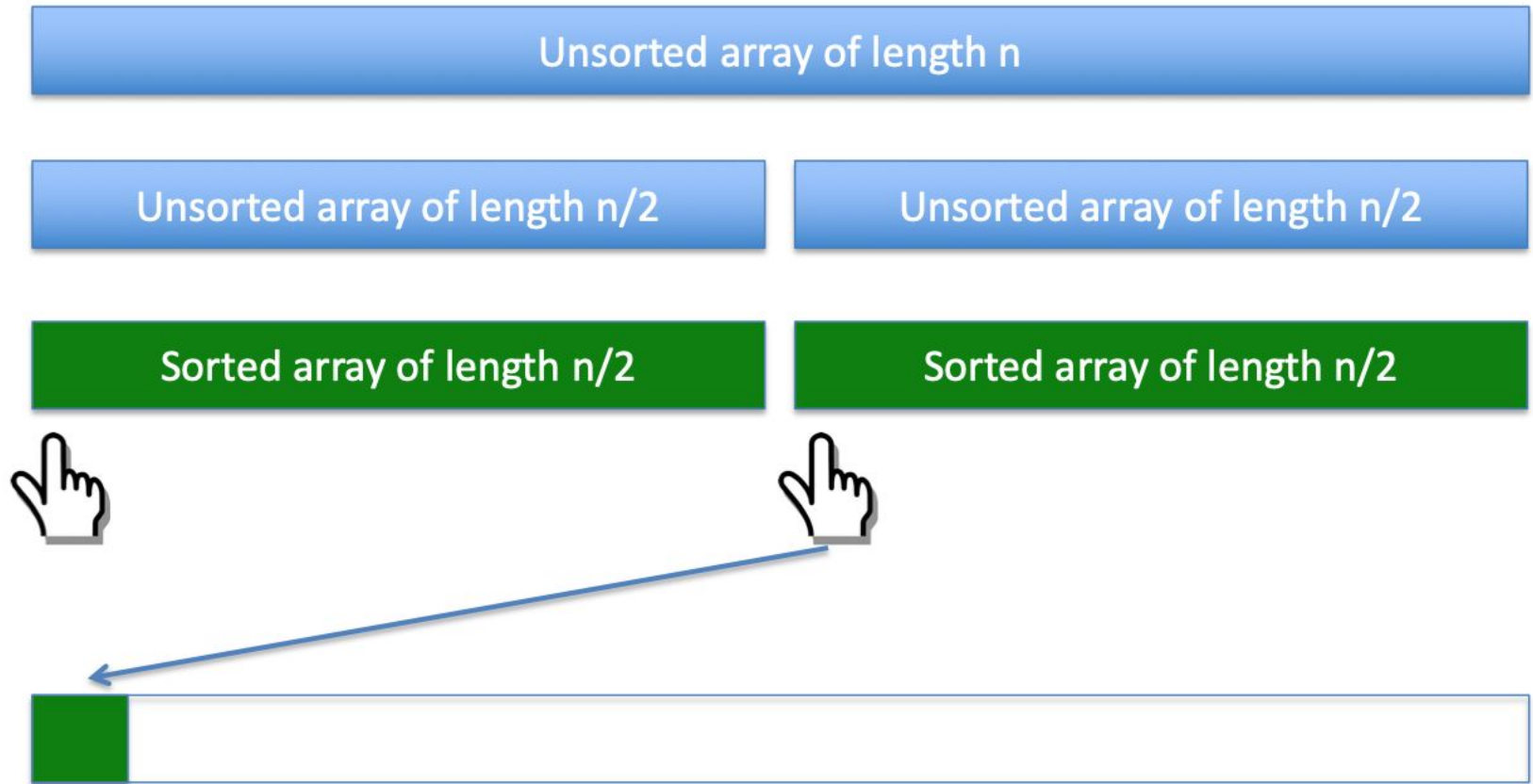
Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

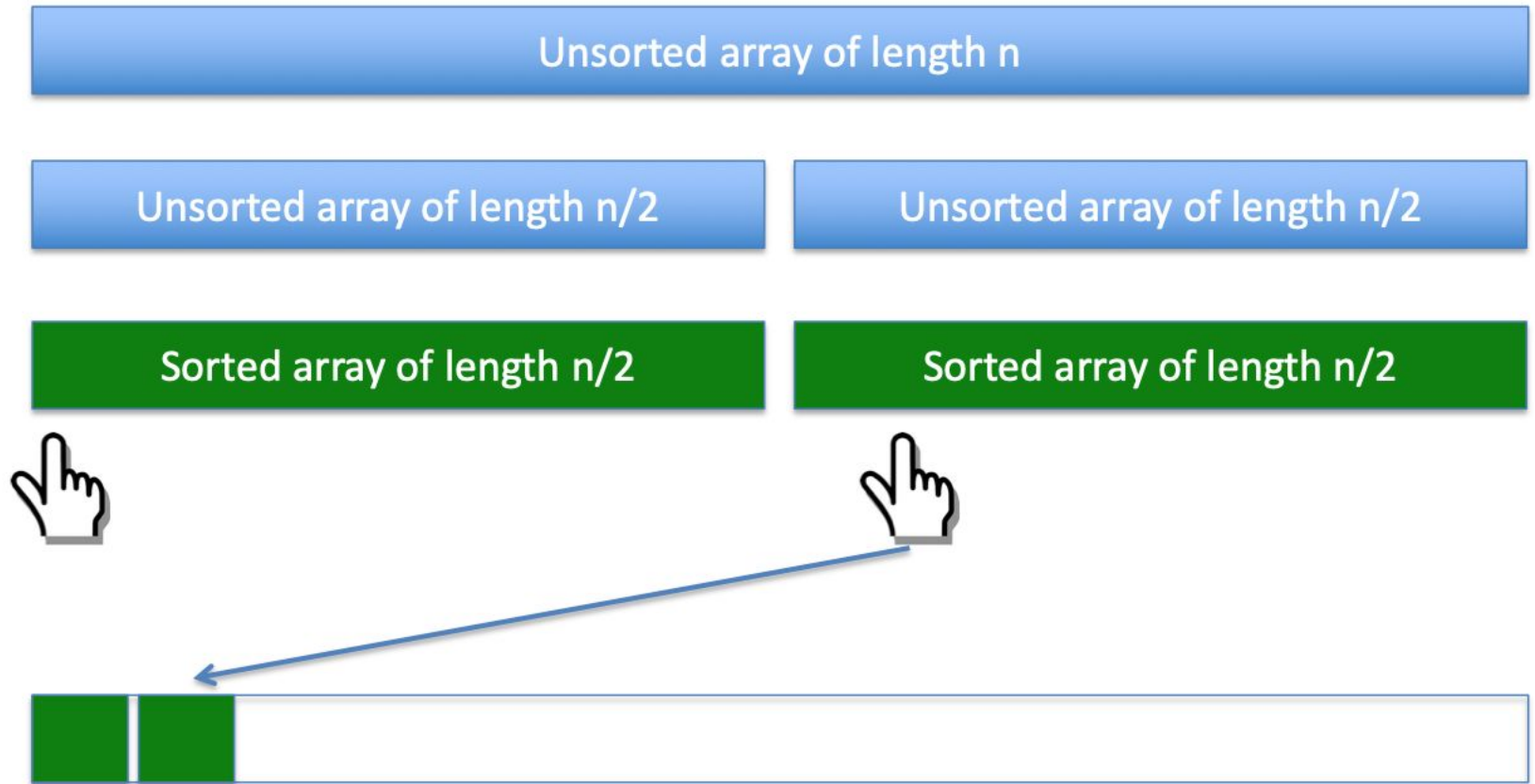
Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort

Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

Sorted array of length $n/2$



Mergesort

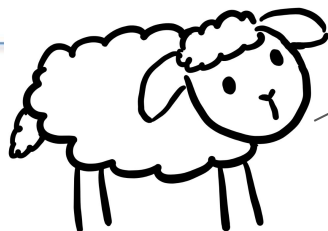
Unsorted array of length n

Unsorted array of length $n/2$

Unsorted array of length $n/2$

Sorted array of length $n/2$

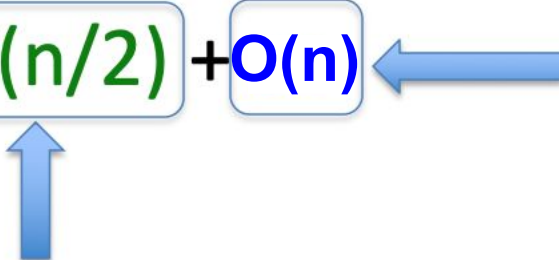
Sorted array of length $n/2$



How long does it take
to merge two sorted
arrays, each of length
 $n/2$?

Recurrences and Running Times

$T(n)$ = worst case running time of mergesort on input of length n

$$T(n) = 2T(n/2) + O(n)$$


Two recursive calls on problems of size $n/2$

Merge two arrays of size $n/2$

Solving Recurrences

The Master Theorem

Let $T(n)$ be the following recurrence, where $k \geq 1$, $b > 1$, $d \geq 0$, $w \geq 0$ are constants (they do not depend on n):

$$T(n) = kT(n/b) + O(n^d \log^w n)$$

You can
use this as
a black box

Then:

$$T(n) = \begin{cases} O(n^d \log^w n) & \text{if } k < b^d \\ O(n^d \log^{w+1} n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d \end{cases}$$



For Mergesort: $k=2$, $b=2$, $d=1$, $w=0 \Rightarrow O(n \log n)$.

Hermit crabs sorting themselves



Another example of divide and conquer:

Integer Multiplication

Input: Two n -digit positive integers x, y

Output: The product $x \cdot y$

“Primitive operations” that can be done in constant time:

- add or multiply two single-digit numbers
- “shift” a number (i.e. add a 0 to the end)

The Grade-School Algorithm

		2	0	3
	×	3	7	6
	1	2	1	8
1	4	2	1	0
6	0	9	0	0
7	6	3	2	8

What is the running time?



An algorithm designer's mantra

“Perhaps the most important principle for the good algorithm designer is to refuse to be content.”

- Aho, Hopcroft, Ullman, *The Design and Analysis of Computer Algorithms* (1974)